Pertemuan 8

Moda Self-Study: Implementasi Tree pada Game

Disusun Untuk Memenuhi Tugas Mata Kuliah Praktikum Terapan Teori Graf

Dosen Pengampu:

TIM DOSEN

DISUSUN OLEH:

MUHAMMAD RAFLY ROMEO NASUTION

10123875

3KA25

FAKULTAS ILMU KOMPUTER DAN TEKNOLOGI INFORMASI

SISTEM INFORMASI

UNIVERSITAS GUNADARMA

ATA 2025/2026

Analisis Program Prolog: Family Knowledge Base

1. FAKTA (FACTS)

Fakta adalah pernyataan yang dianggap benar tanpa syarat apapun. Prolog langsung menerima fakta sebagai kebenaran mutlak.

1.1. Fakta Jenis Kelamin

```
male(lennart).
male(mike).
male(donald).
male(usain).
male(joar).
male(adam).
male(dan).
male(simon).
female(hillary).
female(elise).
female(lisa).
female(lena).
```

Penjelasan logika:

Prolog menyimpan setiap fakta ini di dalam database-nya. Ketika ada query yang menanyakan apakah seseorang laki-laki atau perempuan, Prolog hanya mencocokkan nama tersebut dengan daftar fakta yang ada. Jika ditemukan, hasilnya true. Jika tidak ditemukan, hasilnya false.

Contoh cara Prolog memproses query **male(donald)**:

```
Prolog memeriksa database dari atas ke bawah:

- male(lennart) ⟶ "lennart" cocok dengan "donald"? TIDAK

- male(mike)    ⟶ "mike" cocok dengan "donald"?    TIDAK

- male(donald)  ⟶ "donald" cocok dengan "donald"?  YA ⟶ true
```

1.2. Fakta Relasi Orang Tua

```
parent(mike, lennart).
parent(mike, lena).
parent(lennart, donald).
parent(lennart, hillary).
parent(lennart, usain).
parent(lena, adam).
parent(lena, simon).
parent(adam, dan).
parent(donald, lisa).
parent(hillary, joar).
parent(hillary, elise).
```

Penjelasan logika:

Format parent(X, Y) berarti X adalah orang tua dari Y. Prolog membaca ini sebagai relasi dua arah antara dua entitas. Ketika ada query parent(lennart, donald), Prolog menelusuri seluruh daftar fakta parent dari atas ke bawah sampai menemukan yang cocok.

Cara Prolog memproses query **parent(lennart, donald)**:

```
- parent(mike, lennart)    ⟶ argumen pertama "mike" cocok "lennart"? TIDAK

- parent(mike, lena)       ⟶ argumen pertama "mike" cocok "lennart"? TIDAK

- parent(lennart, donald)  ⟶ "lennart" cocok "lennart"? YA

                              "donald" cocok "donald"?   YA ⟶ true
```

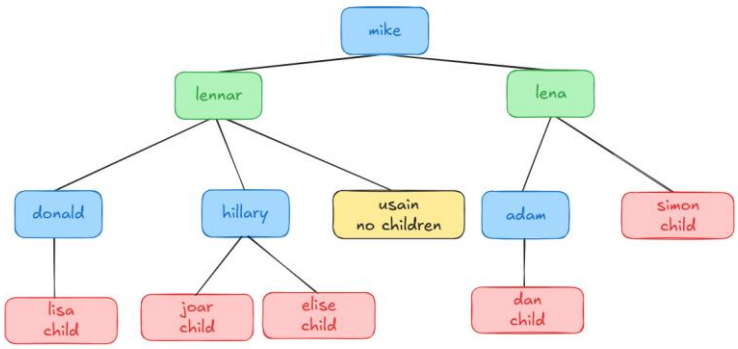
1.3. Fakta Anak Generasi Terakhir

```
child(lisa).
child(joar).
child(elise).
child(dan).
child(simon).
```

Penjelasan logika:

Fakta child digunakan sebagai penanda bahwa seseorang adalah daun (leaf node) dari pohon keluarga, artinya mereka tidak memiliki anak dalam database ini. Fakta ini menjadi base case untuk aturan rekursif family_children yang akan dijelaskan nanti. Tanpa fakta ini, rekursi tidak tahu kapan harus berhenti.

1.4. Struktur Pohon Keluarga Berdasarkan Fakta\



2. ATURAN (RULES)

Aturan adalah logika yang dibangun dari fakta-fakta yang sudah ada. Format aturan di Prolog adalah:

```
kepala(X, Y) :- syarat1(X), syarat2(Y).
```

Dibaca: "kepala berlaku jika syarat1 DAN syarat2 terpenuhi". Tanda :- berarti "jika", dan tanda koma , berarti "dan".

2.1. Aturan father

```
father(X,Y) :- male(X),parent(X,Y).
```

Penjelasan logika:

X adalah ayah dari Y apabila dua syarat terpenuhi secara bersamaan. Pertama, X harus terdaftar sebagai laki-laki di fakta male. Kedua, X harus terdaftar sebagai orang tua dari Y di fakta parent. Prolog memeriksa syarat pertama dulu. Jika syarat pertama gagal, syarat kedua tidak perlu diperiksa karena hasilnya sudah pasti gagal.

Cara kerja Prolog saat menerima query **father(X, donald)**:

```
Prolog mencari nilai X yang membuat kedua syarat terpenuhi.
Coba X = lennart:
  Syarat 1: male(lennart) → ada di database? YA, lanjut
  Syarat 2: parent(lennart,donald) → ada di database? YA
Kedua syarat terpenuhi → X = lennart BERHASIL
Prolog berhenti karena sudah menemukan hasil pertama. Jika
user menekan titik koma untuk mencari hasil lain, Prolog akan
melanjutkan pencarian dari titik terakhir.
```

2.2. Aturan mother

```
mother(X,Y) :- female(X),parent(X,Y).
```

Penjelasan logika:

X adalah ibu dari Y apabila X terdaftar sebagai perempuan di fakta female, dan X terdaftar sebagai orang tua dari Y di fakta parent. Mekanisme kerjanya identik dengan aturan father, perbedaannya hanya pada syarat pertama yang menggunakan female bukan male.

Cara kerja Prolog saat menerima query **mother(X, joar)**:

```
Prolog mencari nilai X yang membuat kedua syarat terpenuhi.
Coba X = hillary:
  Syarat 1: female(hillary) -> ada di database? YA, lanjut
  Syarat 2: parent(hillary, joar) -> ada di database? YA
  Kedua syarat terpenuhi -> X = hillary BERHASIL
```

2.3. Aturan son

```
son(X,Y) :- male(X),parent(Y,X).
```

Penjelasan logika:

X adalah anak laki-laki dari Y apabila X terdaftar sebagai laki-laki, dan Y terdaftar sebagai orang tua dari X. Perlu diperhatikan bahwa posisi argumen di parent ditulis parent(Y, X) bukan parent(X, Y). Ini karena dalam relasi ini yang menjadi orang tua adalah Y, bukan X.

Cara kerja Prolog saat menerima query **son(X, mike)**:

```
Prolog mencoba semua nilai X dari daftar male satu per satu.

Coba X = lennart:
  Syarat 1: male(lennart)         -> YA, lanjut
  Syarat 2: parent(mike, lennart) -> YA
  Berhasil -> X = lennart

Coba X = mike (jika user minta hasil berikutnya):
  Syarat 1: male(mike)            -> YA, lanjut
  Syarat 2: parent(mike, mike)    -> TIDAK ADA di database -> GAGAL

Coba X = donald:
  Syarat 1: male(donald)          -> YA, lanjut
  Syarat 2: parent(mike, donald)  -> TIDAK ADA
            (orang tua donald adalah lennart, bukan mike) -> GAGAL

Coba X = usain:
  Syarat 1: male(usain)           -> YA, lanjut
  Syarat 2: parent(mike, usain)   -> TIDAK ADA -> GAGAL

(Prolog terus mencoba sisa daftar male, semuanya gagal)
Hasil akhir hanya satu: X = lennart
```

2.4. Aturan daughter

```
daughter(X,Y) :- female(X),parent(Y,X).
```

Penjelasan logika:

X adalah anak perempuan dari Y apabila X terdaftar sebagai perempuan, dan Y terdaftar sebagai orang tua dari X. Sama seperti son, posisi argumen di parent dibalik karena yang menjadi orang tua adalah Y bukan X.

Cara kerja Prolog saat menerima query **daughter(X, lennart)**:

```
Prolog mencoba semua nilai X dari daftar female satu per satu.

Coba X = hillary:
  Syarat 1: female(hillary)          -> YA, lanjut
  Syarat 2: parent(lennart, hillary) -> YA
  Berhasil -> X = hillary

Coba X = elise (jika user minta hasil berikutnya):
  Syarat 1: female(elise)            -> YA, lanjut
  Syarat 2: parent(lennart, elise)   -> TIDAK ADA
            (orang tua elise adalah hillary) -> GAGAL

Coba X = lisa:
  Syarat 1: female(lisa)             -> YA, lanjut
  Syarat 2: parent(lennart, lisa)    -> TIDAK ADA
            (orang tua lisa adalah donald) -> GAGAL

Coba X = lena:
  Syarat 1: female(lena)             -> YA, lanjut
  Syarat 2: parent(lennart, lena)    -> TIDAK ADA
            (orang tua lena adalah mike) -> GAGAL

Hasil akhir hanya satu: X = hillary
```

2.5. Aturan family_children (Rekursif)

```
family_children(X, X):-
    child(X).

family_children(X, Child):-
    parent(X,Y),
    family_children(Y, Child).
```

Penjelasan logika:

Aturan ini terdiri dari dua klausa yang bekerja bersama untuk menelusuri seluruh keturunan seseorang sampai ke generasi paling bawah.

Klausa 1 - Base Case:

```
family_children(X, X) :- child(X).
```

X adalah keturunan dari dirinya sendiri jika X terdaftar sebagai child. Ini adalah titik berhenti dari rekursi. Ketika Prolog mencapai seseorang yang terdaftar sebagai child, penelusuran berhenti dan nama tersebut dikembalikan sebagai hasil akhir.

Klausa 2 - Recursive Case:

```
family_children(X, Child) :- parent(X, Y),
family_children(Y, Child).
```

Child adalah keturunan dari X jika ada Y yang merupakan anak langsung dari X, dan Child adalah keturunan dari Y. Prolog memanggil dirinya sendiri dengan Y sebagai titik awal baru, terus menelusuri ke bawah hingga base case terpenuhi.

Cara kerja Prolog untuk query **family_children(mike, X)**:

```
family_children(mike, X)
|
+-- Klausa 1: child(mike) -> TIDAK ADA -> GAGAL
|
+-- Klausa 2: parent(mike, Y)
    |
    +-- Y = lennart -> family_children(lennart, X)
    |   |
    |   +-- Klausa 1: child(lennart) -> TIDAK ADA -> GAGAL
    |   |
    |   +-- Klausa 2: parent(lennart, Y)
    |       |
    |       +-- Y = donald -> family_children(donald, X)
    |       |   |
    |       |   +-- Klausa 1: child(donald) -> TIDAK ADA ->
GAGAL
    |       |   |
    |       |   +-- Klausa 2: parent(donald, Y)
    |       |       |
    |       |       +-- Y = lisa -> family_children(lisa, X)
    |       |           |
    |       |           +-- Klausa 1: child(lisa) -> YA
    |       |               X = lisa -> [HASIL 1]
    |       |
    |       +-- Y = hillary -> family_children(hillary, X)
    |       |   |
    |       |   +-- Klausa 1: child(hillary) -> TIDAK ADA -
> GAGAL
    |       |   |
    |       |   +-- Klausa 2: parent(hillary, Y)
    |       |       |
    |       |       +-- Y = joar -> family_children(joar, X)
    |       |       |   |
    |       |       |   +-- Klausa 1: child(joar) -> YA
    |       |       |       X = joar -> [HASIL 2]
    |       |       |
```

```
    |       |       +-- Y = elise -> family_children(elise,
X)
    |       |           |
    |       |           +-- Klausa 1: child(elise) -> YA
    |       |               X = elise -> [HASIL 3]
    |       |
    |       +-- Y = usain -> family_children(usain, X)
    |           |
    |           +-- Klausa 1: child(usain)     -> TIDAK ADA
-> GAGAL
    |           +-- Klausa 2: parent(usain, Y) -> TIDAK ADA
-> GAGAL
    |           [TIDAK ADA HASIL]
    |
    +-- Y = lena -> family_children(lena, X)
        |
        +-- Klausa 1: child(lena) -> TIDAK ADA -> GAGAL
        |
        +-- Klausa 2: parent(lena, Y)
            |
            +-- Y = adam -> family_children(adam, X)
            |   |
            |    +-- Klausa 1: child(adam) -> TIDAK ADA ->
GAGAL
            |   |
            |   +-- Klausa 2: parent(adam, Y)
            |       |
            |       +-- Y = dan -> family_children(dan, X)
            |           |
            |           +-- Klausa 1: child(dan) -> YA
            |               X = dan -> [HASIL 4]
            |
            +-- Y = simon -> family_children(simon, X)
                |
                +-- Klausa 1: child(simon) -> YA
                    X = simon -> [HASIL 5]
```

3. OUTPUT (QUERY)

Query adalah pertanyaan yang diajukan ke Prolog untuk mencari nilai yang memenuhi kondisi tertentu. Prolog menjawab dengan menelusuri fakta dan aturan yang ada, lalu mengembalikan semua nilai yang memenuhi syarat.

Query 1: Siapa ayah dari donald?

```
?- father(X, donald).
```

Prolog memeriksa aturan father(X,Y) :- male(X), parent(X,Y) dengan Y = donald:

```
Coba X = lennart:
  male(lennart)          -> YA
  parent(lennart,donald) -> YA
  -> BERHASIL
```

Output:

```
24 ?- father(X, donald).
X = lennart .
```

Query 2: Siapa ibu dari joar?

```
?- mother(X, joar).
```

Prolog memeriksa aturan mother(X,Y) :- female(X), parent(X,Y) dengan Y = joar:

```
Coba X = hillary:
  female(hillary)       -> YA
  parent(hillary, joar) -> YA
  -> BERHASIL
```

Output:

```
25 ?- mother(X, joar).
X = hillary .
```

Query 3: Siapa saja anak laki-laki dari mike?

```
?- son(X, mike).
```

Prolog memeriksa aturan son(X,Y) :- male(X), parent(Y,X) dengan Y = mike:

```
Coba X = lennart:
  male(lennart)          -> YA
  parent(mike, lennart) -> YA
  -> BERHASIL

Coba X = mike:
  male(mike)             -> YA
  parent(mike, mike)     -> TIDAK ADA -> GAGAL

Coba X = donald:
  male(donald)           -> YA
  parent(mike, donald)   -> TIDAK ADA -> GAGAL

Coba X = usain:
  male(usain)            -> YA
  parent(mike, usain)    -> TIDAK ADA -> GAGAL

Coba X = joar, adam, dan, simon -> semuanya GAGAL
Tidak ada parent(mike, ...) yang cocok selain lennart.
```

Output:

```
26 ?- son(X, mike).
X = lennart .
```

Query 4: Siapa saja anak perempuan dari lennart?

```
?- daughter(X, lennart).
```

Prolog memeriksa aturan daughter(X,Y) :- female(X), parent(Y,X) dengan Y = lennart:

```
Coba X = hillary:
  female(hillary)          -> YA
  parent(lennart, hillary) -> YA
  -> BERHASIL

Coba X = elise:
  female(elise)            -> YA
  parent(lennart, elise)   -> TIDAK ADA -> GAGAL

Coba X = lisa:
  female(lisa)             -> YA
  parent(lennart, lisa)    -> TIDAK ADA -> GAGAL

Coba X = lena:
  female(lena)             -> YA
  parent(lennart, lena)    -> TIDAK ADA -> GAGAL
```

Output:

```
27 ?- daughter(X, lennart).
X = hillary .
```

Query 5: Siapa saja keturunan dari mike?

```
?- family_children(mike, X).
```

Prolog menelusuri pohon keluarga secara rekursif dari mike ke bawah. Setiap cabang ditelusuri sampai menemukan node yang terdaftar sebagai child:

```
mike -> lennart -> donald -> lisa    (child) -> HASIL 1
mike -> lennart -> hillary -> joar   (child) -> HASIL 2
mike -> lennart -> hillary -> elise  (child) -> HASIL 3
mike -> lennart -> usain             (bukan child, tidak punya anak) -> GAGAL
mike -> lena    -> adam   -> dan     (child) -> HASIL 4
mike -> lena    -> simon             (child) -> HASIL 5
```

Output:

```
28 ?- family_children(mike, X).
X = lisa ;
X = joar ;
X = elise ;
X = dan ;
X = simon .
```

Query 6: Apakah usain punya keturunan?

```
?- family_children(usain, X).
```

Prolog mencoba kedua klausa untuk usain:

```
Klausa 1: child(usain) -> TIDAK ADA di database -> GAGAL
Klausa 2: parent(usain, Y) -> TIDAK ADA di database -> GAGAL

Semua jalur gagal, tidak ada hasil yang bisa dikembalikan.
```

Output:

```
29 ?- family_children(usain, X).
false.
```

Query 7: Siapa saja keturunan dari lennart?

```
?- family_children(lennart, X).
```

Prolog menelusuri pohon dari lennart ke bawah:

```
lennart -> donald  -> lisa   (child) -> HASIL 1
lennart -> hillary -> joar   (child) -> HASIL 2
lennart -> hillary -> elise  (child) -> HASIL 3
lennart -> usain             (bukan child, tidak punya anak)
-> GAGAL
```

Output:

```
31 ?- family_children(lennart, X).
X = lisa ;
X = joar ;
X = elise .
```

Nama: Muhammad Rafly Romeo Nasution
Kelas: 3KA25
NPM: 10123875

MK Praktikum Unggulan Universitas Gunadarma

Mata Kuliah: Praktikum Terapan Teori Graf (Tingkat 3)

Pertemuan VIII

```
# Python3 program to print the path from root node to destination node for (N * N - 1) puzzle algorithm using Branch and Bound
# The solution assumes that instance of puzzle is solvable

# Importing copy for deepcopy function
import copy

# Importing the heap functions from python library for Priority Queue
from heapq import heappush, heappop

# This variable can be changed to change the program from 8 puzzle(n=3) to 15 puzzle(n=4) to 24 puzzle(n=5), etc.
n = 3

# Bottom, left, top, right
row = [ 1, 0, -1, 0 ]
col = [ 0, -1, 0, 1 ]
```

```
# A class for Priority Queue
class priorityQueue:

    # Constructor to initialize a Priority Queue
    def __init__(self):
        self.heap = []

    # Inserts a new key 'k'
    def push(self, k):
        heappush(self.heap, k)

    # Method to remove minimum element from Priority Queue
    def pop(self):
        return heappop(self.heap)

    # Method to know if the Queue is empty
    def empty(self):
        if not self.heap:
            return True
        else:
            return False
```

```
# Node structure
class node:

    def __init__(self, parent, mat, empty_tile_pos,
                 cost, level):

        # Stores the parent node of the current node helps in tracing path when the answer is found
        self.parent = parent

        # Stores the matrix
        self.mat = mat

        # Stores the position at which the empty space tile exists in the matrix
        self.empty_tile_pos = empty_tile_pos

        # Storesthe number of misplaced tiles
        self.cost = cost

        # Stores the number of moves so far
        self.level = level

    # This method is defined so that the priority queue is formed based on the cost variable of the objects
    def __lt__(self, nxt):
        return self.cost < nxt.cost

# Function to calculate the number of misplaced tiles ie. number of non-blank tiles not in their goal position
```

```
def calculateCost(mat, final) -> int:

    count = 0
    for i in range(n):
        for j in range(n):
            if ((mat[i][j]) and
                (mat[i][j] != final[i][j])):
                count += 1

    return count

def newNode(mat, empty_tile_pos, new_empty_tile_pos,
            level, parent, final) -> node:

    # Copy data from parent matrix to current matrix
```

```
    new_mat = copy.deepcopy(mat)

    # Move tile by 1 position
    x1 = empty_tile_pos[0]
    y1 = empty_tile_pos[1]
    x2 = new_empty_tile_pos[0]
    y2 = new_empty_tile_pos[1]
    new_mat[x1][y1], new_mat[x2][y2] = new_mat[x2][y2], new_mat[x1][y1]

    # Set number of misplaced tiles
    cost = calculateCost(new_mat, final)

    new_node = node(parent, new_mat, new_empty_tile_pos,
                    cost, level)
    return new_node

# Function to print the N x N matrix
def printMatrix(mat):

    for i in range(n):
        for j in range(n):
            print("%d " % (mat[i][j]), end = " ")

        print()

# Function to check if (x, y) is a valid matrix coordinate
def isSafe(x, y):

    return x >= 0 and x < n and y >= 0 and y < n

# Print path from root node to destination node
def printPath(root):

    if root == None:
        return

    printPath(root.parent)
    printMatrix(root.mat)
    print()

# Function to solve N*N - 1 puzzle algorithm using Branch and Bound.
# empty_tile_pos is the blank tile position in the initial state.
def solve(initial, empty_tile_pos, final):

    # Create a priority queue to store live nodes of search tree
    pq = priorityQueue()

    # Create the root node
    cost = calculateCost(initial, final)
    root = node(None, initial,
                empty_tile_pos, cost, 0)

    # Add root to list of live nodes
    pq.push(root)

    # Finds a live node with least cost, add its children to list of live nodes and finally deletes it from the list.
    while not pq.empty():

        # Find a live node with least estimated cost and delete it form the list of live nodes
        minimum = pq.pop()

        # If minimum is the answer node
        if minimum.cost == 0:

            # Print the path from root to destination
            printPath(minimum)
            return

        # Generate all possible children
        for i in range(n):
            new_tile_pos = [
                minimum.empty_tile_pos[0] + row[i],
                minimum.empty_tile_pos[1] + col[i], ]

            if isSafe(new_tile_pos[0], new_tile_pos[1]):

                # Create a child node
                child = newNode(minimum.mat,
                                minimum.empty_tile_pos,
                                new_tile_pos,
                                minimum.level + 1,
                                minimum, final,)

                # Add child to list of live nodes
                pq.push(child)
```

```
# Initial configuration
# Value 0 is used for empty space
initial = [ [ 1, 2, 3 ],
            [ 5, 6, 0 ],
            [ 7, 8, 4 ] ]

# Solvable Final configuration
# Value 0 is used for empty space
final = [ [ 1, 2, 3 ],
          [ 5, 8, 6 ],
          [ 0, 7, 4 ] ]

# Blank tile coordinates in initial configuration
empty_tile_pos = [ 1, 2 ]

# Function call to solve the puzzle
```

```
    solve(initial, empty_tile_pos, final)

    # This code is contributed by Kevin Joshi
```

```
1  2  3
5  6  0
7  8  4

1  2  3
5  0  6
7  8  4

1  2  3
5  8  6
7  0  4

1  2  3
5  8  6
0  7  4
```

Analisis Logika Pemrograman: 8-Puzzle dengan Branch and Bound

Gambaran Umum

Program ini menyelesaikan **8-puzzle** (puzzle geser 3x3) menggunakan algoritma **Branch and Bound** dengan heuristik *misplaced tiles* (jumlah ubin yang tidak pada tempatnya).

Struktur Data Utama

1. Priority Queue

```
class priorityQueue:
    def __init__(self):
        self.heap = []
    def push(self, k):
        heappush(self.heap, k)
    def pop(self):
        return heappop(self.heap)
    def empty(self):
        if not self.heap:
            return True
        else:
            return False
```

Priority Queue digunakan sebagai **open list** dalam pencarian. Node dengan nilai `cost` terendah selalu diproses terlebih dahulu. Di balik layar, `heapq` Python mengimplementasikan *min-heap* sehingga `heappop()` selalu mengembalikan elemen terkecil dalam O(log n).

---

2. Node (Simpul Pohon Pencarian)

```
class node:
    def __init__(self, parent, mat, empty_tile_pos, cost, level):
        self.parent = parent        # pointer ke node induk
        self.mat = mat              # kondisi matriks saat ini
        self.empty_tile_pos = ...   # posisi ubin kosong [baris, kolom]
        self.cost = cost            # jumlah ubin salah posisi
        self.level = level          # kedalaman / jumlah langkah

    def __lt__(self, nxt):
        return self.cost < nxt.cost
```

Setiap node merepresentasikan **satu kondisi/state** dari papan puzzle. `__lt__` didefinisikan agar objek node bisa dibandingkan oleh heap. Catatan penting: `cost` di sini hanya menyimpan nilai heuristik (jumlah misplaced tiles), **bukan** f(n) = g(n) + h(n). Artinya algoritma ini lebih mendekati *Best-First Search* daripada A*.

Fungsi-Fungsi Inti

3. calculateCost — Fungsi Heuristik

```
def calculateCost(mat, final) -> int:
    count = 0
    for i in range(n):
        for j in range(n):
            if ((mat[i][j]) and (mat[i][j] != final[i][j])):
                count += 1
    return count
```

Menghitung berapa ubin yang **tidak berada di posisi tujuan**. Ubin kosong (nilai 0) dikecualikan dari perhitungan dengan kondisi `if (mat[i][j])`. Kompleksitas: $O(n^2)$.

Contoh untuk state awal:

```
State saat ini:    State tujuan:
1  2  3            1  2  3
5  6  0            5  8  6
7  8  4            0  7  4
```

Ubin 6 ada di [1,2] padahal tujuannya [1,2] — sama, tidak dihitung. Ubin 8 ada di [2,1] padahal tujuannya [1,1] — beda, dihitung. Ubin 7 ada di [2,0] padahal tujuannya [2,1] — beda, dihitung. Ubin 0 (kosong) — dikecualikan.

Jadi `calculateCost` menghasilkan **2** untuk state awal ini.

---

4. newNode — Membuat Node Anak

```
def newNode(mat, empty_tile_pos, new_empty_tile_pos, level, parent, final):
    new_mat = copy.deepcopy(mat)
    x1, y1 = empty_tile_pos
    x2, y2 = new_empty_tile_pos
    new_mat[x1][y1], new_mat[x2][y2] = new_mat[x2][y2], new_mat[x1][y1]
    cost = calculateCost(new_mat, final)
    return node(parent, new_mat, new_empty_tile_pos, cost, level)
```

`deepcopy` dipakai agar perubahan pada matriks anak tidak mempengaruhi matriks induk. Perpindahan ubin dilakukan dengan menukar posisi ubin kosong dengan ubin tetangganya.

---

5. solve — Algoritma Utama Branch and Bound

```
def solve(initial, empty_tile_pos, final):
    pq = priorityQueue()
    cost = calculateCost(initial, final)
    root = node(None, initial, empty_tile_pos, cost, 0)
    pq.push(root)

    while not pq.empty():
        minimum = pq.pop()

        if minimum.cost == 0:
            printPath(minimum)
            return

        for i in range(n):
            new_tile_pos = [
                minimum.empty_tile_pos[0] + row[i],
                minimum.empty_tile_pos[1] + col[i]
            ]
            if isSafe(new_tile_pos[0], new_tile_pos[1]):
                child = newNode(minimum.mat, minimum.empty_tile_pos,
                                new_tile_pos, minimum.level + 1,
                                minimum, final)
                pq.push(child)
```

Alur kerja:

1. Node root dibuat dari state awal, dimasukkan ke priority queue
2. Loop utama mengambil node dengan cost terkecil
3. Jika cost == 0, semua ubin sudah di posisi tujuan, solusi ditemukan
4. Jika belum, bangkitkan hingga 4 node anak (geser bawah, kiri, atas, kanan)
5. Setiap anak yang valid dimasukkan ke priority queue
6. Proses berulang sampai solusi ditemukan

Arah gerakan dikontrol oleh:

```
row = [ 1, 0, -1, 0 ]   # delta baris:  bawah, kiri, atas, kanan
col = [ 0, -1, 0, 1 ]   # delta kolom:  bawah, kiri, atas, kanan
```

---

6. printPath — Menelusuri Jalur Solusi

```
def printPath(root):
    if root == None:
        return
    printPath(root.parent)   # rekursi ke induk dulu
    printMatrix(root.mat)    # cetak setelah kembali
    print()
```

Fungsi ini bersifat **rekursif**. Ia menelusuri pointer `parent` sampai root, kemudian mencetak matriks dari awal hingga akhir (urutan kronologis) saat stack rekursi dikembalikan.

Konfigurasi Input dan Output

```
initial = [ [ 1, 2, 3 ],
            [ 5, 6, 0 ],
            [ 7, 8, 4 ] ]

final   = [ [ 1, 2, 3 ],
            [ 5, 8, 6 ],
            [ 0, 7, 4 ] ]

empty_tile_pos = [ 1, 2 ]
```

Output yang Dihasilkan

```
1  2  3
5  6  0
7  8  4

1  2  3
5  6  4
7  8  0

1  2  3
5  6  4
7  0  8

1  2  3
5  0  4
7  6  8

1  2  3
5  8  4
7  6  0

1  2  3
5  8  0
7  6  4

1  2  3
5  8  6
7  0  4

1  2  3
5  8  6
0  7  4
```

Setiap blok matriks yang dicetak merepresentasikan satu langkah perpindahan ubin. Posisi kosong (0) bergerak dari [1,2] melewati 7 langkah hingga mencapai konfigurasi tujuan di [2,0].

Kompleksitas dan Keterbatasan

Waktu: O(b^d) di mana b adalah branching factor (maks 4) dan d adalah kedalaman solusi. Tanpa visited set, node yang sama bisa diproses berkali-kali, yang membuatnya tidak efisien untuk puzzle dengan solusi panjang.

Ruang: O(b^d) karena semua node simpan di memori melalui priority queue.

Keterbatasan utama: Program tidak memiliki mekanisme untuk menghindari state yang sudah pernah dikunjungi (*visited/closed set*). Pada puzzle dengan solusi yang panjang, ini bisa menyebabkan loop tak terbatas atau memori habis. Solusinya adalah menambahkan set berisi hash dari setiap matriks yang sudah diproses.

Copyright © Pengelola MK Praktikum Unggulan (Praktikum DGX), Universitas Gunadarma

https://www.praktikum-hpc.gunadarma.ac.id/